САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЁТ

ПО ДИСЦИПЛИНЕ «БЭК-ЭНД РАЗРАБОТКА»

по теме:

ЛР5: МОНИТОРИНГ МИКРОСЕРВИСОВ СРЕДСТВАМИ GRAFANA (СЕРВИС АРЕНДЫ НЕДВИЖИМОСТИ)

Проверил: Добряков Д. И.

подпись, дата

Выполнил: Субботин Александр, БР1.1

подпись, дата

Санкт-Петербург 2026

1 Задача

Настроить сервис мониторинга микросервисов средствами Grafana. Реализовать дашборд, покрывающий:

1. общее потребление CPU, disk, RAM, Network на ВМ;
2. healthcheck для всех микросервисов, показывающий, все ли сервисы доступны;
3. просмотр логов со всех контейнеров, запущенных на ВМ.

Дополнительно:

1. alert на изменение healthcheck с отправкой почтовых уведомлений (2 балла);
2. интеграция Sentry: self-hosted (4 балла) или облачного (2 балла).

Стек: Grafana 13.0, Prometheus 3.13, node-exporter 1.12, blackbox-exporter 0.29, Loki 3.6, Promtail 3.6, Mailpit 1.31, sentry-go 0.50. Мониторится приложение из ЛР4 (`labs/lab4-5`), развёрнутое на ВМ `13.143.182.131` (Debian 12, 1 vCPU, 1.9 ГБ RAM); Grafana доступна по адресу `https://rental.zzisst.space/grafana/`.

2 Ход работы

2.1 Архитектура мониторинга

Стек мониторинга описан в отдельном файле `docker-compose.monitoring.yaml` и подключается к приложению через `COMPOSE_FILE` в `.env`, поэтому всё поднимается одной командой `docker compose up -d` (и тем же `deploy.sh` из ЛР4). Сервисы мониторинга находятся в той же сети `rental_backend`, что и приложение, и обращаются к микросервисам по их DNS-именам.

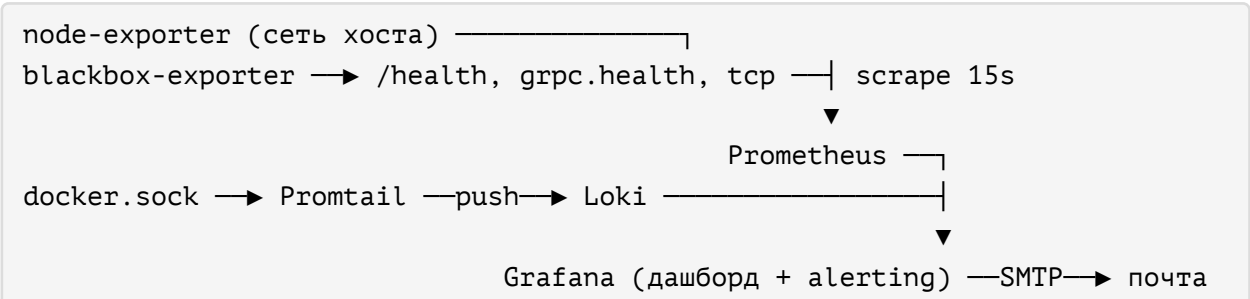


Таблица 1. Компоненты стека мониторинга

Компонент	Назначение
node-exporter	метрики ВМ. Запущен с `network_mode: host`, `pid: host` и корнем ФС хоста в `/host` (read-only), поэтому видит CPU, память, диски и сетевые интерфейсы именно ВМ, а не контейнера
blackbox-exporter	активные проверки: HTTP `GET /health` (код 200 и `"status":"ok"` в теле), gRPC Health Checking Protocol, TCP-подключение
Prometheus	сбор метрик раз в 15 с, хранение 15 дней / 2 ГБ
Promtail	обнаруживает все контейнеры через Docker API (`docker_sd_configs`) и отправляет их stdout/stderr в Loki
Loki	хранение логов (TSDB, schema v13, ретеншн 7 дней)
Grafana	datasource'ы, дашборд и алерты описаны как код (provisioning) — после `docker compose up` всё готово без ручной настройки

Наружу опубликована только Grafana на `127.0.0.1:3000`; извне она доступна через nginx по пути `/grafana/` (`GF_SERVER_SERVE_FROM_SUB_PATH=true`).

2.2 Healthcheck всех микросервисов

Для property- и booking-service, у которых основное API — gRPC, в сервер добавлен стандартный сервис `grpc.health.v1.Health`. Его статус раз в 10 секунд обновляется по `db.Ping()`, так что `NOT_SERVING` означает реальную недоступность сервиса, а не только «процесс жив»:

```
func RegisterHealth(s *grpc.Server, db *sql.DB, service string, interval
time.Duration) {
    hs := health.NewServer()
    healthpb.RegisterHealthServer(s, hs)
    update := func() {
        st := healthpb.HealthCheckResponse_SERVING
        if err := db.PingContext(ctx); err != nil {
            st = healthpb.HealthCheckResponse_NOT_SERVING
        }
        hs.SetServingStatus(service, st)
    }
    ...
}
```

В Prometheus настроены три job'а blackbox-проверок. У каждой цели есть метки `service` и `probe`, по которым строятся панели и алерты:

```
- job_name: health-http
  metrics_path: /probe
```

```
    params: { module: [http_health] }
    static_configs:
      - targets: ["http://api-gateway:8080/health"]
        labels: { service: api-gateway, probe: http }
      - targets: ["http://user-service:8081/health"]
        labels: { service: user-service, probe: http }
      # ... property-service, booking-service, chat-service
    relabel_configs:
      - { source_labels: [__address__], target_label: __param_target }
      - { source_labels: [__param_target], target_label: instance }
      - { target_label: __address__, replacement: blackbox-exporter:9115 }
  - job_name: health-grpc      # property-service:9082, booking-service:9083
  - job_name: health-tcp       # postgres:5432, rabbitmq:5672
```

Итого 9 проверок: HTTP `/health` пяти сервисов, gRPC health двух сервисов, TCP БД и брокера.

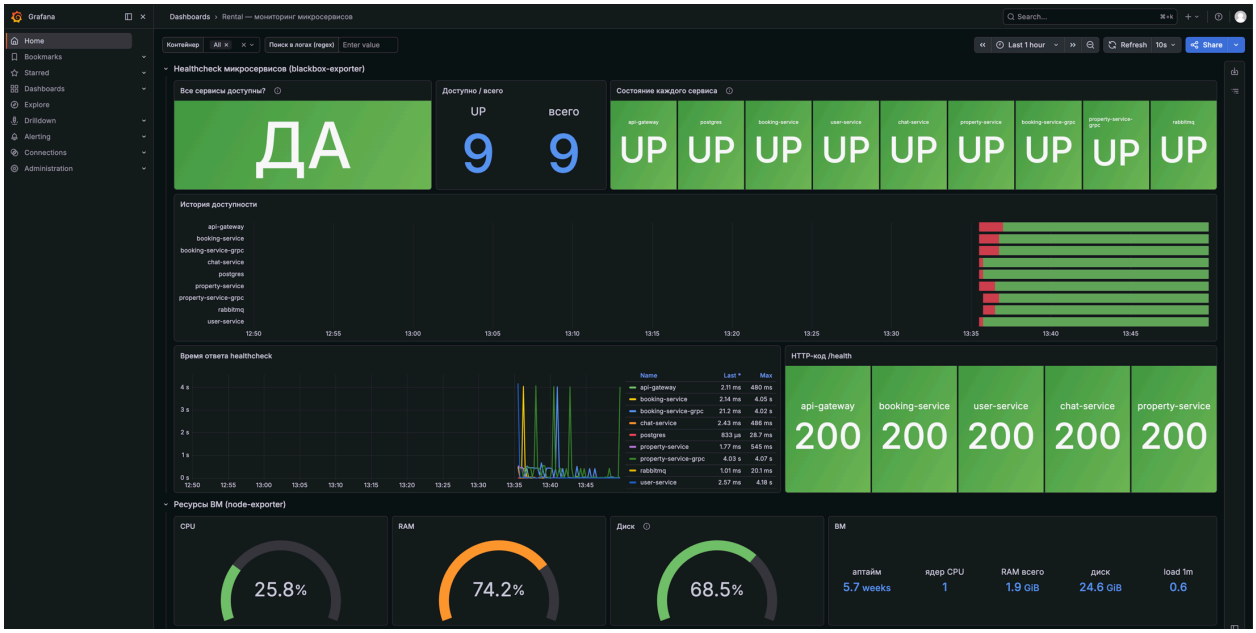


Рисунок 1. Дашборд на сервере: блок healthcheck (общий статус, UP/всего, плитка на каждый сервис, история доступности — красный участок в 13:35 соответствует первому запуску, пока сервисы поднимались; время ответа, HTTP-код) и начало блока ресурсов ВМ

Панель «Все сервисы доступны?» — это `min(probe_success)`: 1 (ДА), только если проходят все проверки. «История доступности» — state timeline по `max by (service) (probe_success)`.

2.3 Ресурсы ВМ

Таблица 2. Основные запросы блока ресурсов ВМ

Панель	**PromQL**
CPU, %	`100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[$__rate_interval])))`
RAM, %	`100 * (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)`
Диск, %	`max(100 * (1 - node_filesystem_avail_bytes / node_filesystem_size_bytes))` по реальным ФС (без tmpfs/overlay)
Сеть	`rate(node_network_receive_bytes_total[...]) * 8` и `...transmit...` по физическим интерфейсам (без `lo`, `veth*`, `docker*`, `br-*`)
Диск I/O	`rate(node_disk_read_bytes_total[...])`, `rate(node_disk_written_bytes_total[...])`

Prometheus находится в docker-сети, а node-exporter — в сети хоста, поэтому для Prometheus имя `node-exporter` направлено на шлюз docker (`extra_hosts: node-exporter:host-gateway`), а в `ufw` порт 9100 открыт только для docker-подсетей.

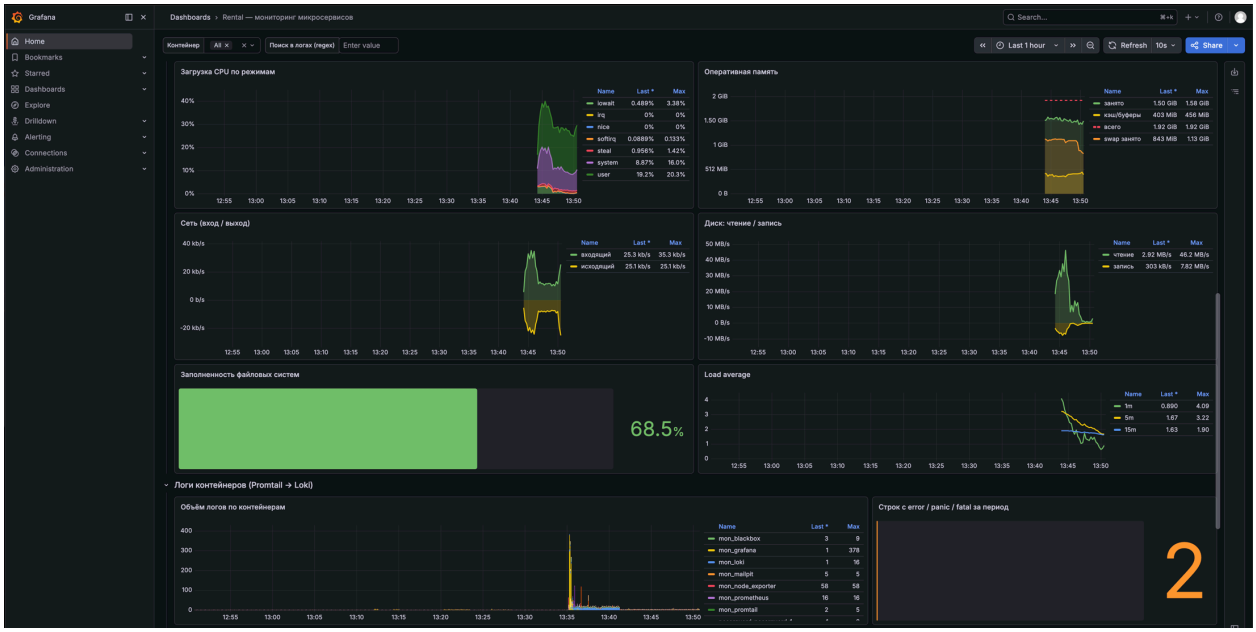


Рисунок 2. Ресурсы ВМ на сервере: CPU по режимам, память (видно использование swap), сеть, дисковый I/O, заполненность ФС, load average; начало блока логов

На сервере видно реальное состояние ВМ: 1 ядро, 1.9 ГиБ RAM, диск 24.6 ГиБ заполнен на 68.5%, аптайм 5.7 недели.

2.4 Логи всех контейнеров

Promtail подключается к `docker.sock` и сам находит все контейнеры на ВМ — и приложения, и мониторинга, и любые другие (например, self-hosted Sentry). Из docker-меток формируются метки Loki `container`, `service` (compose-сервис), `project`, `stream`. Из строк access-лога gin регуляркой извлекаются HTTP-статус и метод — как structured metadata, чтобы не увеличивать кардинальность меток:

```
scrape_configs:
  - job_name: docker
    docker_sd_configs:
      - host: unix:///var/run/docker.sock
    relabel_configs:
      - { source_labels: [__meta_docker_container_name], regex: "/(.*)",
target_label: container }
      - { source_labels:
[__meta_docker_container_label_com_docker_compose_service], target_label:
service }
    pipeline_stages:
      - regex:
          expression: '^\[GIN\].*?\|\s*(?P<status>\d{3})\s*\|.*?\|\s*(?
P<method>[A-Z]+)\s+"'
      - structured_metadata: { status: , method: }
```

На дашборде есть переменные «Контейнер» (мультивыбор, по умолчанию — все) и «Поиск в логах» (regex), график объёма логов по контейнерам и счётчик строк с `error|panic|fatal`.

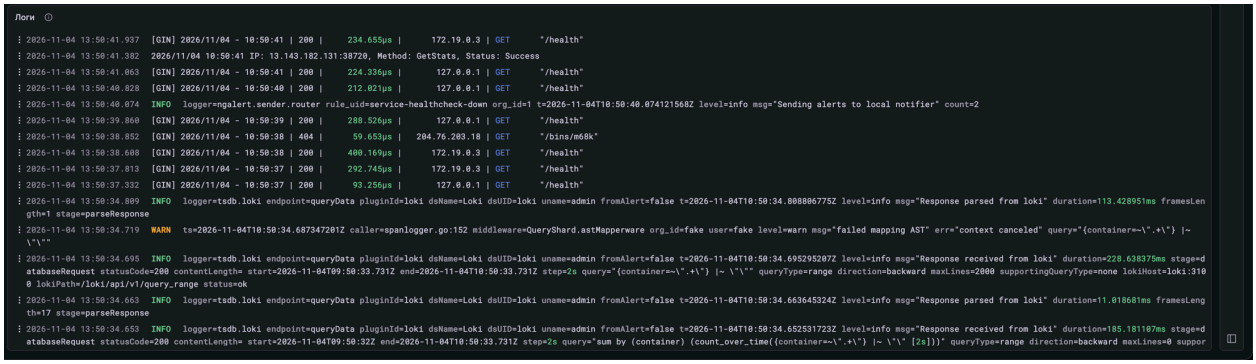


Рисунок 3. Лента логов всех контейнеров ВМ: access-логи gin (в т.ч. сканер из интернета, получивший 404), логи VPN-ноды, Grafana, Loki

В выборку попадают все контейнеры на ВМ, включая не относящиеся к приложению (панель VPN, её TimescaleDB и pgbouncer), — Promtail не фильтрует их по compose-проекту.

2.5 Доп. задание 1: алерт на изменение healthcheck (почта)

Алертинг Grafana настроен через provisioning (`monitoring/grafana/provisioning/alerting`):

- правило «Healthcheck сервиса не проходит»: `min by (service, probe, instance) (probe_success) < 1` в течение 30 секунд, вычисляется каждые 10 секунд. Правило многомерное — для каждого сервиса создаётся отдельный экземпляр алерта;
- точка контакта `email-admin`: адрес из переменной `ALERT_EMAIL`, тема письма 🔴 `DOWN: <сервис>` при срабатывании и 🟢 `UP: <сервис>` при восстановлении (`disableResolveMessage: false`) — то есть письмо приходит на любое изменение состояния;
- политика уведомлений: группировка по `alertname, service`, повтор раз в 4 часа.

SMTP задаётся переменными `SMTP_*` в `.env`. При локальной отладке письма ловит Mailpit (тестовый SMTP-сервер с веб-интерфейсом), на сервере используется SMTP Mail.ru:

```
SMTP_HOST=smtp.mail.ru:465          # порт 465 — неявный TLS
SMTP_USER=zzisst@mail.ru
SMTP_PASSWORD=<пароль для внешнего приложения>
SMTP_FROM=zzisst@mail.ru            # Mail.ru требует, чтобы отправитель
совпадал с логином
SMTP_STARTTLS_POLICY=NoStartTLS
ALERT_EMAIL=zzisst@mail.ru
```

Проверка на сервере: `docker stop rental_chat_service` в 14:22:39 → письмо «🔴 DOWN: chat-service» в 14:23 (30 с `for` + интервал вычисления + `group_wait`); `docker start` в 14:25:10 → письмо «🟢 UP: chat-service» в 14:25.

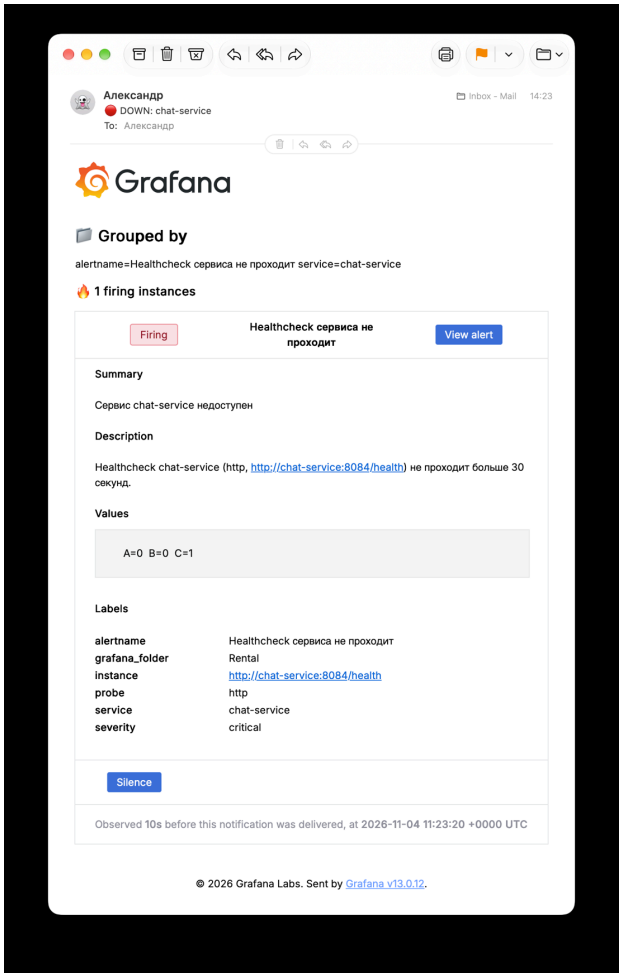


Рисунок 4. Письмо при падении: Firing, сервис, тип проверки, адрес healthcheck, метки

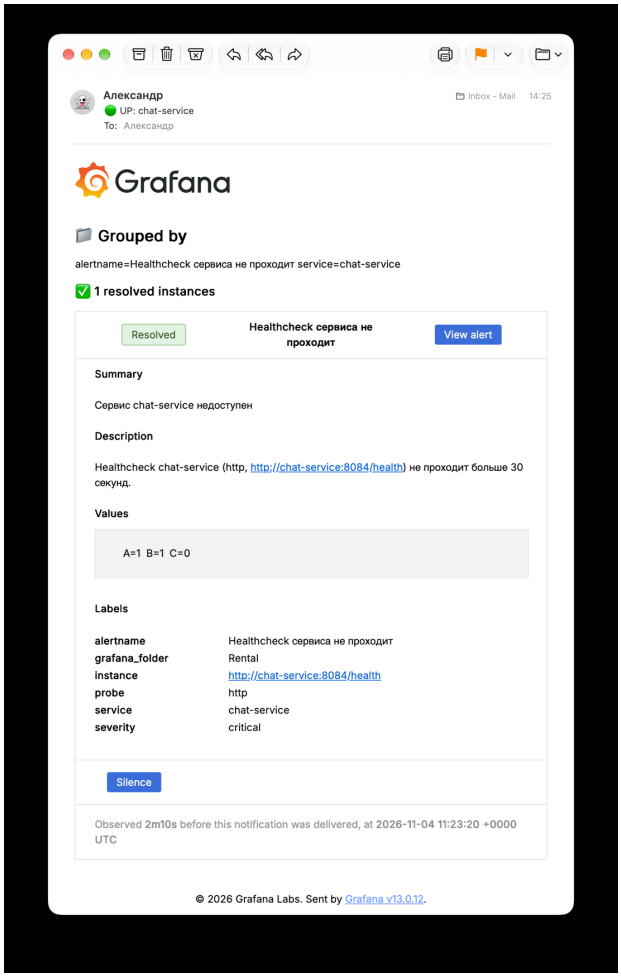


Рисунок 5. Письмо при восстановлении: Resolved

2.6 Доп. задание 2: Sentry

Во все пять сервисов добавлен пакет `internal/observability` на базе `sentry-go`. SDK настраивается только переменными окружения, поэтому один и тот же образ работает и с self-hosted Sentry, и с sentry.io; при пустом `SENTRY_DSN` все вызовы — no-op.

```
sentry.Init(sentry.ClientOptions{
    Dsn:              os.Getenv("SENTRY_DSN"),
    Environment:      getEnv("SENTRY_ENVIRONMENT", "local"),
    Release:          os.Getenv("SENTRY_RELEASE"),   // git sha из CI (Д36)
    ServerName:       service,
    AttachStacktrace: true,
    TracesSampleRate: tracesRate,
})
```

Что отправляется в Sentry:

- **gin** (user, chat, gateway, health-серверы): `sentrygin` перехватывает паники (`Repanic: true` — ответ 500 по-прежнему формирует `gin.Recovery`), а собственный middleware отправляет ответы со статусом 5xx, которые хендлеры вернули без паники (например, при недоступной БД). Gateway отправляет только свои 502–504, чтобы не дублировать ошибки сервисов;
- **gRPC** (property, booking): unary-интерцептор отправляет ошибки с кодами `Internal/Unknown/DataLoss/Unavailable` с тегами `grpc.method` и `grpc.code` и перехватывает паники, превращая их в `codes.Internal` (раньше паника в обработчике роняла весь сервис);
- `GET /debug/sentry` (только при `SENTRY_DEBUG_ROUTE=true`, через nginx закрыт) — тестовая паника.

Логика интерцептора покрыта unit-тестами (`grpc_test.go`): восстановление после паники и классификация кодов. Доставка событий проверена локальным приёмником envelope API:

```
EVENT /api/1/envelope/ server: user-service env: localtest tags:
{'service': 'user-service'}
      msg: sentry test panic from user-service
EVENT /api/1/envelope/ server: property-service env: localtest
      tags: {'grpc.code': 'Internal', 'grpc.method': '/
propertypublic.PropertyService/ListProperties',
              'service': 'property-service'}
      exc: *status.Error rpc error: code = Internal desc = internal error
```

Выбор варианта. Для self-hosted подготовлен скрипт `deploy/sentry/install-self-hosted.sh`: он разворачивает официальный getsentry/self-hosted 26.10.0 в облегчённом профиле `errors-only`, привязывает Sentry к `127.0.0.1:9000`, прописывает `system.url-prefix` и `CSRF_TRUSTED_ORIGINS`, создаёт администратора и публикует Sentry через nginx на отдельном порту (`deploy/nginx/sentry.conf`: Sentry не поддерживает работу в под-пути). Однако даже `errors-only` по `install/_min-requirements.sh` требует 2 CPU и 7000 МБ RAM, а у используемой ВМ 1 vCPU и 1.9 ГБ, из которых свободно около 400 МБ. Поэтому интеграция выполнена с облачным sentry.io: в нём создан проект `go-gin` (платформа Go / Gin), его DSN записан в `SENTRY_DSN` в `.env` на сервере, после чего контейнеры пересозданы деплоем из CI (ДЗ6). Код сервисов от варианта не зависит.

Проверка на сервере. В 14:22:11 база данных приложения была остановлена на 5 секунд, и за это время к API через nginx отправлены запросы

к `/properties`, `/bookings`, `/favorites`, `/users/me/properties` (gRPC-сервисы) и `/chats`, `/users/me` (HTTP-сервисы) — все вернули 500. Sentry сгруппировал события в Issues и прислал уведомления о новых проблемах на почту:

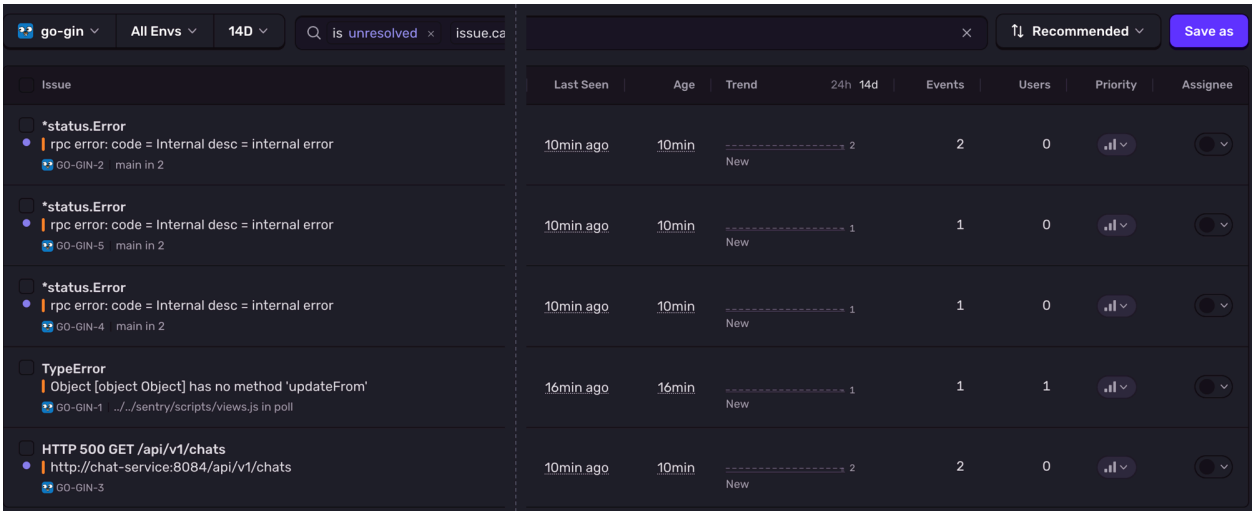


Рисунок 6. Issues проекта go-gin в sentry.io: три Issue `*status.Error ... Internal` от gRPC-сервисов (4 события) и `HTTP 500` от HTTP-сервисов (2 события); GO-GIN-1 (`TypeError ... views.js`) — пример ошибки, созданный самим Sentry при онбординге («View Sample Error»), к приложению не относится; пустая середина таблицы вырезана

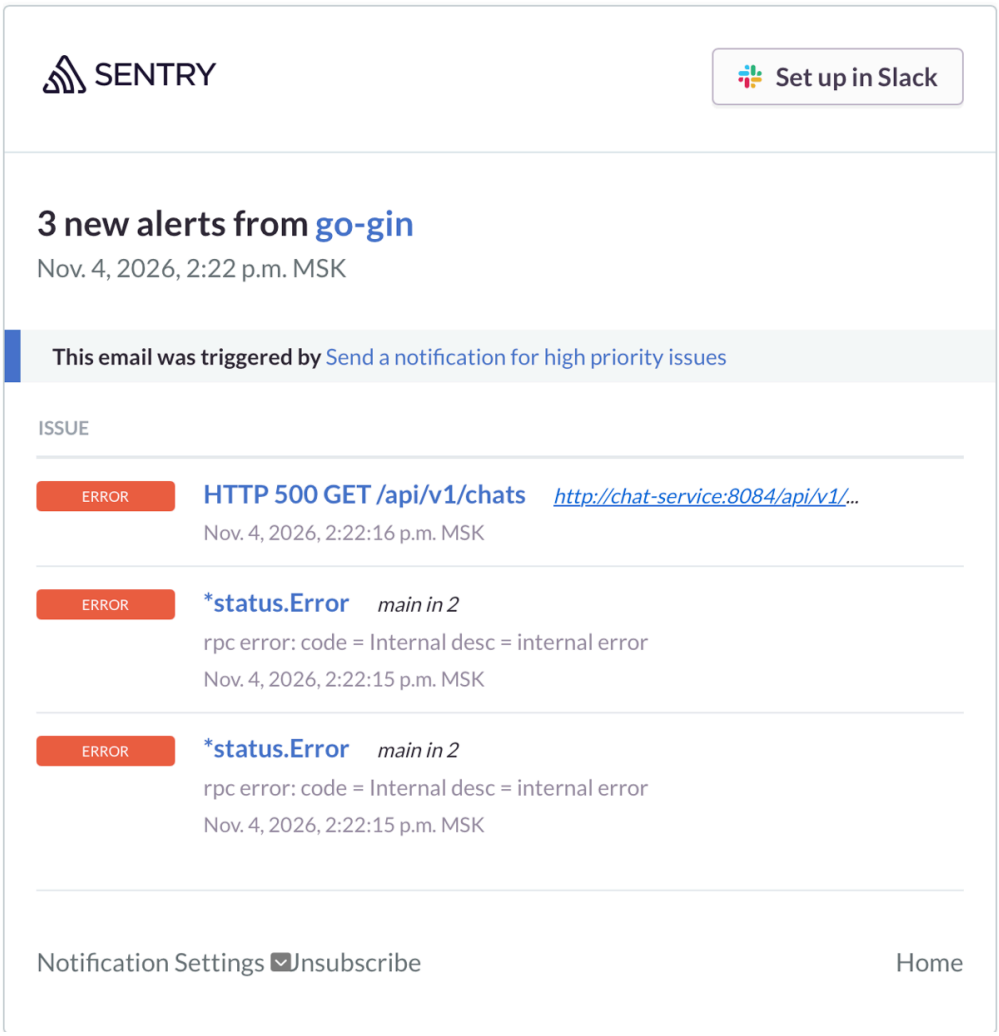


Рисунок 7. Уведомление Sentry о новых Issues: HTTP 500 от chat-service (middleware gin) и ошибки gRPC `Internal` от property- и booking-service

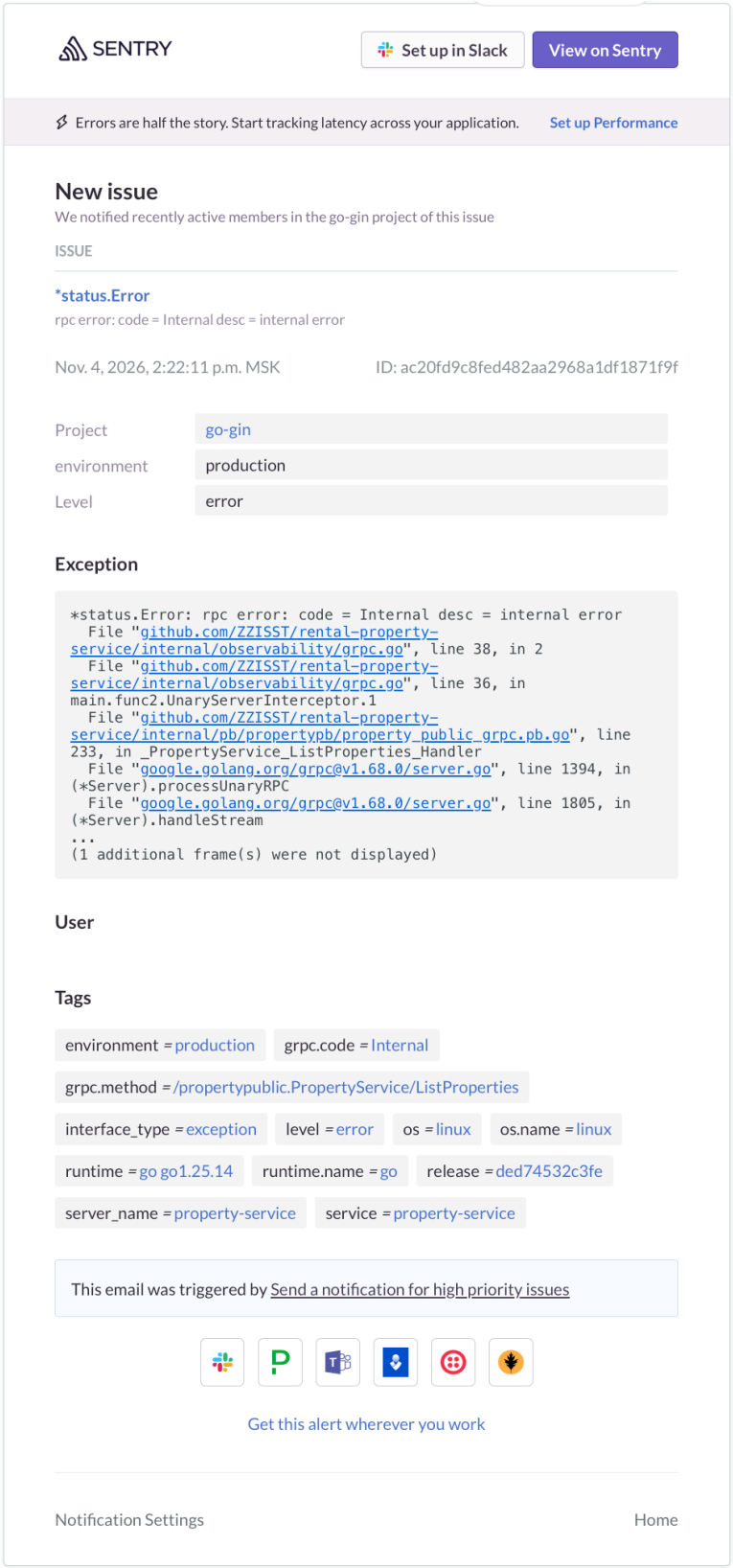


Рисунок 8. Событие из property-service: стектрейс через `UnaryServerInterceptor`, теги `service`, `grpc.method`, `grpc.code`, `release` (sha коммита из CI) и `environment=production`

У события из gRPC-сервиса видны метод (`/propertypublic.PropertyService/ListProperties`), код ошибки и версия

релиза `ded74532c3fe` — тот же sha, который CI использовал как тег образа, — поэтому по ошибке сразу понятно, в какой версии и в каком методе она возникла.

3 Вывод

Развёрнут стек мониторинга Grafana + Prometheus + Loki + Promtail, полностью описанный как код: datasource'ы, дашборд и алерты поднимаются автоматически вместе с приложением. Дашборд показывает ресурсы ВМ (CPU, RAM, диск, сеть), состояние всех микросервисов по HTTP-, gRPC- и TCP-проверкам и логи всех контейнеров с фильтрацией и поиском. Для gRPC-сервисов реализован стандартный протокол `grpc.health.v1`, учитывающий доступность БД. Дополнительно настроены почтовые уведомления о падении и восстановлении каждого сервиса и интеграция всех сервисов с Sentry (паники, ответы 5xx, ошибки gRPC) — облачным sentry.io, так как ресурсов ВМ недостаточно для self-hosted (для него подготовлен скрипт установки).